# DATA STRUCTURES & ALGORITHMS

# LAB MANUAL — PYTHON EDITION

Python translations of the 10 experiments in the supplied C++ lab manual

Includes Aim, Algorithm / Logic, and commented Python source code for each experiment.

Experiments 1–10

# Experiment 1: Array Operations (Insert / Delete / Traverse)

**Aim:** Perform insertion, deletion, and traversal operations on an array using 0-based indexing.

## Algorithm / Logic

1. Read the initial list of elements.

2. For insertion, validate the index and insert the value at that position.

3. For deletion, validate the index and remove the element.

4. Display the current elements; repeat until Exit is selected.

## Python Code

```
# Array Operations: insertion, deletion, and traversal

arr = list(map(int, input("Enter elements separated by spaces: ").split()))

while True:
    print("\\n--- Array Operations Menu ---")
    print("1. Insert\\n2. Delete\\n3. Display\\n4. Exit")
    choice = input("Enter choice: ").strip()

    if choice == "1":
        pos = int(input("Enter 0-based position and value: ").split()[0])
        value = int(input("Enter value to insert: "))
        if 0 <= pos <= len(arr):
            arr.insert(pos, value)
            print(f"Inserted {value} at position {pos}.")
        else:
            print("Invalid position.")

    elif choice == "2":
        pos = int(input("Enter 0-based position to delete: "))
        if 0 <= pos < len(arr):
            removed = arr.pop(pos)
            print(f"Deleted {removed} from position {pos}.")
        else:
            print("Invalid position.")

    elif choice == "3":
        print("Array elements:", *arr)

    elif choice == "4":
        print("Exiting array operations.")
        break

    else:
        print("Invalid choice. Try again.")
```

# Experiment 2: Linear Search

**Aim:** Search for an element in a list by checking each position sequentially.

## Algorithm / Logic

1. Read the elements and the target value.

2. Check each element from index 0 onward.

3. If an element matches the target, report its index and stop.

4. If no match is found, report that the element was not found.

## Python Code

```
# Linear Search
# Time complexity: O(n)

arr = list(map(int, input("Enter elements separated by spaces: ").split()))
key = int(input("Enter value to search: "))

for i, value in enumerate(arr):
    if value == key:
        print(f"Element found at index {i} (0-based).")
        break
else:
    print("Element not found.")
```

# Experiment 3: Binary Search

**Aim:** Search for an element in a sorted list using divide-and-conquer.

## Algorithm / Logic

1. Input elements in sorted order and the target value.
2. Set low to 0 and high to the last index.
3. Calculate mid and compare the middle element with the target.
4. Narrow the search to the left or right half until found or the range is empty.

## Python Code

```
# Binary Search (input must be sorted)
# Time complexity: O(log n)

arr = list(map(int, input("Enter sorted elements separated by spaces: ").split()))
key = int(input("Enter value to search: "))

low, high = 0, len(arr) - 1

while low <= high:
    mid = low + (high - low) // 2

    if arr[mid] == key:
        print(f"Element found at index {mid} (0-based).")
        break
    elif key < arr[mid]:
        high = mid - 1
    else:
        low = mid + 1
else:
    print("Element not found.")
```

# Experiment 4: Selection Sort

**Aim:** Sort a list by repeatedly selecting the minimum element and placing it at the beginning of the unsorted portion.

## Algorithm / Logic

1. For each position except the last, assume that position contains the minimum.
2. Search the remaining unsorted portion for a smaller element.
3. Swap the minimum element with the element at the current position.
4. Repeat until the list is sorted.

## Python Code

```
# Selection Sort
# Time complexity: O(n^2)

arr = list(map(int, input("Enter elements separated by spaces: ").split()))

for i in range(len(arr) - 1):
    min_index = i

    for j in range(i + 1, len(arr)):
        if arr[j] < arr[min_index]:
            min_index = j

    arr[i], arr[min_index] = arr[min_index], arr[i]

print("Sorted list:", *arr)
```

# Experiment 5: Bubble Sort

**Aim:** Sort a list by repeatedly comparing adjacent elements and swapping them when they are out of order.

## Algorithm / Logic

1. Run passes over the list.
2. Compare each adjacent pair in the unsorted portion.
3. Swap the pair if the left element is greater than the right element.
4. After each pass, the largest remaining element is placed at the end.

## Python Code

```
# Bubble Sort
# Time complexity: O(n^2); stable when only swapping out-of-order pairs

arr = list(map(int, input("Enter elements separated by spaces: ").split()))

for i in range(len(arr) - 1):
    for j in range(len(arr) - i - 1):
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]

print("Sorted list:", *arr)
```

# Experiment 6: Singly Linked List (All Operations)

**Aim:** Implement insertion at the start/end, deletion by value, search, and display for a singly linked list.

## Algorithm / Logic

1. Each node stores data and a reference to the next node.

2. Insertion at the end traverses to the last node; insertion at the start updates the head.

3. Deletion removes the first node containing the target value and reconnects the links.

4. Search checks each node; display traverses and prints all values.

## Python Code

```
# Singly Linked List: insert at start/end, delete, search, display

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


def insert_at_end(head, value):
    new_node = Node(value)
    if head is None:
        return new_node
    current = head
    while current.next is not None:
        current = current.next
    current.next = new_node
    return head


def insert_at_start(head, value):
    new_node = Node(value)
    new_node.next = head
    return new_node


def delete_node(head, value):
    if head is None:
        return None
    if head.data == value:
        return head.next

    previous = head
    current = head.next
    while current is not None and current.data != value:
        previous = current
        current = current.next

    if current is not None:
        previous.next = current.next
    return head


def search(head, key):
    current = head
    while current is not None:
        if current.data == key:
            return True
        current = current.next
    return False


def display(head):
    values = []
    current = head
    while current is not None:
        values.append(str(current.data))
        current = current.next
    print(" -> ".join(values) if values else "List is empty")


head = None
n = int(input("Enter number of nodes to insert: "))
```

```
for _ in range(n):
    value = int(input("Enter value: "))
    head = insert_at_end(head, value)

print("List after insertion at end:", end=" ")
display(head)

head = insert_at_start(head, 5)
print("After inserting 5 at start:", end=" ")
display(head)

value = int(input("Enter value to delete: "))
head = delete_node(head, value)
print("List after deletion:", end=" ")
display(head)

value = int(input("Enter value to search: "))
print("Found" if search(head, value) else "Not Found")
```

# Experiment 7: Doubly Linked List (All Operations)

**Aim:** Implement insertion at the start/end, deletion by value, and forward display for a doubly linked list.

## Algorithm / Logic

1. Each node stores data, a previous reference, and a next reference.

2. Insertion at the end connects the new node to the last node.

3. Insertion at the start updates the old head's previous reference.

4. Deletion reconnects the neighboring nodes; display follows next references.

## Python Code

```
# Doubly Linked List: insert at start/end, delete, display forward

class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None


def insert_at_end(head, value):
    new_node = Node(value)
    if head is None:
        return new_node
    current = head
    while current.next is not None:
        current = current.next
    current.next = new_node
    new_node.prev = current
    return head


def insert_at_start(head, value):
    new_node = Node(value)
    new_node.next = head
    if head is not None:
        head.prev = new_node
    return new_node


def delete_node(head, value):
    current = head
    while current is not None and current.data != value:
        current = current.next
    if current is None:
        return head

    if current.prev is not None:
        current.prev.next = current.next
    else:
        head = current.next

    if current.next is not None:
        current.next.prev = current.prev
    return head


def display(head):
    values = []
    current = head
    while current is not None:
        values.append(str(current.data))
        current = current.next
    print(" <-> ".join(values) if values else "List is empty")


head = None
n = int(input("Enter number of nodes to insert: "))
for _ in range(n):
    head = insert_at_end(head, int(input("Enter value: ")))

print("Doubly linked list:", end=" ")
display(head)
```

```
head = insert_at_start(head, 100)
print("After inserting 100 at start:", end=" ")
display(head)

value = int(input("Enter value to delete: "))
head = delete_node(head, value)
print("After deletion:", end=" ")
display(head)
```

# Experiment 8: Circular Linked List (All Operations)

**Aim:** Implement insertion at the start/end, deletion by value, and display for a circular singly linked list.

## Algorithm / Logic

1. The last node's next reference points back to the head.

2. Insertion at the end connects the new node to the head.

3. Insertion at the start updates the head and the last node's next reference.

4. Deletion handles a single node, the head, and other nodes; display stops when it returns to the head.

## Python Code

```
# Circular Singly Linked List: insert at start/end, delete, display

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


def insert_at_end(head, value):
    new_node = Node(value)
    if head is None:
        new_node.next = new_node
        return new_node

    last = head
    while last.next != head:
        last = last.next
    last.next = new_node
    new_node.next = head
    return head


def insert_at_start(head, value):
    new_node = Node(value)
    if head is None:
        new_node.next = new_node
        return new_node

    last = head
    while last.next != head:
        last = last.next
    new_node.next = head
    last.next = new_node
    return new_node


def delete_node(head, value):
    if head is None:
        print("List is empty.")
        return None

    # One-node list
    if head.data == value and head.next == head:
        return None

    # Deleting the head
    if head.data == value:
        last = head
        while last.next != head:
            last = last.next
        head = head.next
        last.next = head
        return head

    previous = head
    current = head.next
    while current != head and current.data != value:
        previous = current
        current = current.next

    if current == head:
        print("Value not found.")
```

```
        return head

    previous.next = current.next
    return head


def display(head):
    if head is None:
        print("List is empty.")
        return

    values = []
    current = head
    while True:
        values.append(str(current.data))
        current = current.next
        if current == head:
            break
    print("Circular linked list:", " -> ".join(values), "-> (head)")


head = None
n = int(input("Enter number of nodes to insert: "))
for _ in range(n):
    head = insert_at_end(head, int(input("Enter value: ")))

print("List after insertion:", end=" ")
display(head)

head = insert_at_start(head, 999)
print("After inserting 999 at start:", end=" ")
display(head)

value = int(input("Enter value to delete: "))
head = delete_node(head, value)
print("After deletion:", end=" ")
display(head)
```

# Experiment 9: Stack Using Array

**Aim:** Implement stack operations (push, pop, peek, and display) using a fixed-capacity list.

## Algorithm / Logic

1. Maintain top as the index of the top element; initialize it to -1.
2. Push stores a value at the next position unless the stack is full.
3. Pop decreases top unless the stack is empty.
4. Peek returns the top element; display prints from top to bottom.

## Python Code

```
# Stack using a fixed-capacity list
MAX = 100


class Stack:
    def __init__(self):
        self.arr = [None] * MAX
        self.top = -1

    def is_empty(self):
        return self.top == -1

    def is_full(self):
        return self.top == MAX - 1

    def push(self, value):
        if self.is_full():
            print("Stack Overflow")
            return
        self.top += 1
        self.arr[self.top] = value

    def pop(self):
        if self.is_empty():
            print("Stack Underflow")
            return
        self.arr[self.top] = None
        self.top -= 1

    def peek(self):
        if self.is_empty():
            print("Stack is empty")
            return None
        return self.arr[self.top]

    def display(self):
        print(*(self.arr[i] for i in range(self.top, -1, -1)))


stack = Stack()
n = int(input("Enter number of elements to push: "))
for _ in range(n):
    stack.push(int(input("Enter value: ")))

print("Stack (top to bottom):", end=" ")
stack.display()
print("Peek top:", stack.peek())
print("Pop one element")
stack.pop()
print("Stack now:", end=" ")
stack.display()
```

# Experiment 10: Queue Using Array (Linear Queue)

**Aim:** Implement enqueue, dequeue, front/peek, and display operations using a fixed-capacity linear queue.

## Algorithm / Logic

1. Maintain front and rear indices; initialize front to 0 and rear to -1.

2. Enqueue increments rear and stores the value unless the queue is full.

3. Dequeue increments front unless the queue is empty.

4. Peek returns the front element; display prints from front through rear.

## Python Code

```
# Queue using a fixed-capacity list (linear queue)
MAX = 100


class Queue:
    def __init__(self):
        self.arr = [None] * MAX
        self.front = 0
        self.rear = -1

    def is_empty(self):
        return self.rear < self.front

    def is_full(self):
        return self.rear == MAX - 1

    def enqueue(self, value):
        if self.is_full():
            print("Queue Overflow")
            return
        self.rear += 1
        self.arr[self.rear] = value

    def dequeue(self):
        if self.is_empty():
            print("Queue Underflow")
            return
        self.front += 1

    def peek(self):
        if self.is_empty():
            print("Queue is empty")
            return None
        return self.arr[self.front]

    def display(self):
        if self.is_empty():
            print("Queue is empty")
            return
        print(*(self.arr[i] for i in range(self.front, self.rear + 1)))


queue = Queue()
n = int(input("Enter number of elements to enqueue: "))
for _ in range(n):
    queue.enqueue(int(input("Enter value: ")))

print("Queue (front to rear):", end=" ")
queue.display()
print("Front element:", queue.peek())
print("Dequeue one element")
queue.dequeue()
print("Queue now:", end=" ")
queue.display()
```